

(even), we divide it by 2 to get 2 and push a squaring operation to `opStack`. Since `n` is now 2 and even again, we divide it by 2 to get 1 and push another squaring operation to `opStack`. Now that `n` is 1, we are finished with this first step.

To perform the second step, we start the `result` as `a` (which is 6). We pop the `opStack` stack to get a squaring operation, telling the program to set `result` to `result * result` (that is, `result2`) or 36. We pop the next operation off `opStack`, and it is another squaring operation, so the program changes the 36 in `result` to `36 * 36`, or 1296. We pop the last operation off `opStack`, and it is a multiply-by-`a` operation, so we multiply the 1296 in `result` by `a` (which is 6) to get 7776. There are no more operations on `opStack`, so the function is now finished. When we double-check our math, we find that 6^5 is indeed 7,776.

The stack in `opStack` looks like [Figure 2-4](#) as the function call `exponentWithPowerRule(6, 5)` executes.

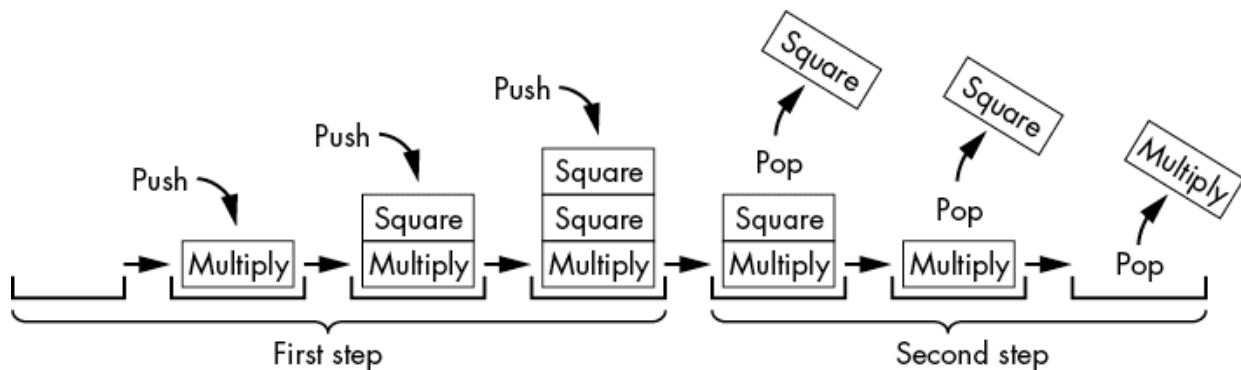


Figure 2-4: The stack in `opStack` during the `exponentWithPowerRule(6, 5)` function call

When you run this code, the output is identical to the other exponent programs:

```
729
1000
2015993900449
```

The iterative exponents function that uses the power rule has the improved performance of the recursive algorithm, while not suffering from the risk of a stack overflow. We might not have thought of this new, improved iterative algorithm without the insights of recursive thinking.